



۸۱۰۱۰۲۳۴۳

۸۱۰۱۰۱۴۹۲

شبکه‌های کامپیوتری
امیرحسین منصوری و علیرضا کریمی



پروژه ۱ فاز ۱

۱ مقدمه

هدف اصلی این پروژه، راه‌اندازی یک محیط توسعه عملی برای P4 و پیاده‌سازی یک سوئیچ لایه ۲ یادگیرنده بود. در حالی که هدف نهایی، شبیه‌سازی ترافیک شبکه با استفاده از Mininet بود، بخش عمده‌ی کار به غلبه بر چالش‌های مجازی‌سازی و شبکه‌ای برای راه‌اندازی محیط P4 روی یک سیستم میزبان ویندوز اختصاص داشت. این گزارش، فرآیند کامل کار را از تبدیل تصویر دیسک و عیب‌یابی خطاهای حیاتی ماشین مجازی تا بازرسی جداول یک سوئیچ P4 در حال اجرا، به طور دقیق شرح می‌دهد.

۲ فاز اول: راه‌اندازی و عیب‌یابی ماشین مجازی

از آنجایی که فایل ارائه‌شده درسی یک تصویر دیسک QEMU با پسوند qcow2 بود و سیستم میزبان من ویندوز با VMware Workstation است، اولین مرحله تبدیل فرمت تصویر بود.

۱.۲ تبدیل تصویر دیسک

از ابزار qemu-img برای تبدیل دیسک به فرمت سازگار با VMware استفاده کردم:

```
qemu-img.exe convert -O vmdk p4-utils-vm.qcow2 p4-utils-vm.vmdk
```

سپس یک ماشین مجازی جدید از نوع «ابونتو ۶۴ بیتی» ایجاد کرده و فایل تبدیل‌شده را به عنوان هارد دیسک SCSI موجود به آن متصل کردم.

۲.۲ خطای حیاتی: خرابی اسنپ‌شات

پس از یک قطعی، هنگام تلاش برای راه‌اندازی مجدد ماشین مجازی با یک خطای بحرانی مواجه شدم که مانع بوت شدن می‌شد. VMware پیام زیر را نشان می‌داد:

”دیسک مجازی والد از زمان ایجاد فرزند تغییر کرده است.“

تحلیل مشکل:

ماشین مجازی یک فایل ”اسنپ‌شات“ (مثلاً p4-vm-000002.vmdk) برای ذخیره تغییرات اخیر ایجاد کرده بود. با این حال، پیوند بین این اسنپ‌شات و دیسک سخت اصلی قطع شده بود که احتمالاً به دلیل خاموش شدن نامناسب ماشین بود.

راه‌حل:

برای بازیابی ماشین مجازی، مجبور شدم:

۱. به تنظیمات ماشین مجازی رفته و حذف گزینه دیسک سخت خراب را انجام دهم.
۲. به پوشه پروژه D:computer network رفته و به صورت دستی فایل اسنپ‌شات خراب (p4-vm-000002.vmdk) را حذف کنم.
۳. فایل اصلی p4-vm.vmdk را مجدداً به عنوان دیسک اصلی اضافه کنم.

۳.۲ خطای حیاتی: قفل شدن فایل

در طول فرآیند عیب‌یابی، با خطای ”File Locked“ نیز مواجه شدم که در آن VMware ادعا می‌کرد فرآیند دیگری در حال استفاده از دیسک است.

راه‌حل:

به دایرکتوری ماشین مجازی رفتم و پوشه‌هایی با پسوند lck را پیدا کردم. حذف این پوشه‌های قفل، فایل را آزاد کرد و به VMware اجازه داد دوباره ماشین را روشن کند.

۳ فاز دوم: رفع مشکلات اتصال شبکه

پس از راه‌اندازی VM، مشکل اصلی عدم اتصال به اینترنت بود که در چند مرحله عیب‌یابی و حل شد:

۱. **تست حالت NAT:** در ابتدا، کارت شبکه روی حالت NAT تنظیم شد. ماشین مجازی یک IP دریافت کرد (192.168.137.50) اما فاقد default gateway بود که منجر به خطای Network is unreachable می‌شد. تلاش برای افزودن دستی gateway نیز به خطای Destination Host Unreachable انجامید که نشان‌دهنده اختلال در سرویس‌های NAT نرم‌افزار VMware روی ویندوز بود.
۲. **تغییر به حالت Bridged:** به عنوان راه‌حل، حالت شبکه به Bridged تغییر یافت تا VM مستقیماً از روتر شبکه IP دریافت کند.

۳. فعال‌سازی کارت شبکه: پس از تغییر حالت، مشخص شد که کارت شبکه VM در حالت DOWN قرار دارد.

۴. حل نهایی: با اجرای دو دستور زیر، کارت شبکه فعال و IP جدید دریافت شد:

```
sudo ip link set ens33 up
```

```
sudo dhclient ens33
```

پس از این مرحله، ping به 8.8.8.8 با موفقیت با صفر درصد packet loss متصل شد و اتصال به اینترنت برقرار گردید.

در شکل ۱ توضیحات دو دستور مربوط به ستاپ کردن شبکه و اختصاص پورت را می‌بینید به واسطه *bridge* است.

```
ub.com
p4@p4:~/p4-tools$ sudo ip link set ens33 up
p4@p4:~/p4-tools$ sudo dhclient ens33
^Z
[2]+  Stopped                  sudo dhclient ens33
p4@p4:~/p4-tools$ sudo dhclient ens33

^Z
[3]+  Stopped                  sudo dhclient ens33
p4@p4:~/p4-tools$ cd
p4@p4:~$ ping 8.8.8.8
ping: connect: Network is unreachable
p4@p4:~$ ping 8.8.8.8
ping: connect: Network is unreachable
p4@p4:~$ sudo ip link set ens33 up
p4@p4:~$ sudo dhclient ens33
^Z
[4]+  Stopped                  sudo dhclient ens33
p4@p4:~$ sudo apt install openssh-server
Reading package lists... Done
Building dependency tree
Reading state information... Done
openssh-server is already the newest version (1:8.2p1-4ubuntu0.9).
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
p4@p4:~$ sudo systemctl enable ssh
Synchronizing state of ssh.service with SysV service script with /lib/systemd/systemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install enable ssh
p4@p4:~$ sudo ufw enable
Firewall is active and enabled on system startup
p4@p4:~$ sudo ufw allow ssh
Rule added
Rule added (v6)
p4@p4:~$ _
```

شکل ۱: resolving the problem of connction to the network

۴ فاز سوم: پیاده‌سازی پروژه

۱.۴ دانلود سورس کد

با بازیابی اینترنت، اقدام به کلون کردن مخزن p4-learning کردم. کلون استاندارد چندین بار با خطای resolve host به دلیل قطع‌های متناوب اتصال، شکست خورد.

راه‌حل:

از روش "کلون سطحی" استفاده کردم تا فقط تاریخچه آخرین commit را دانلود کنم که حجم دانلود را به طور قابل توجهی کاهش داده و از تایم‌اوت جلوگیری کرد:

```
git clone --depth 1 https://github.com/nsg-ethz/p4-learning.git
```

در شکل ۲ می‌توانید ببینید که کلون به درستی انجام شده.

```

Password:
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.4.0-162-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

System information as of Wed 10 Dec 2025 03:17:29 PM UTC

System load: 0.61           Memory usage: 8%   Processes:      371
Usage of /:  60.6% of 42.15GB Swap usage:   0%   Users logged in: 0

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

   https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

11 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection
or proxy settings

Last login: Wed Dec 10 14:33:36 UTC 2025 on tty1
p4@p4:~$ git clone --depth 1 https://github.com/nsg-ethz/p4-learning.git
Cloning into 'p4-learning'...
fatal: unable to access 'https://github.com/nsg-ethz/p4-learning.git/': Could not resolve host: gith
ub.com
p4@p4:~$

```

شکل ۲: efficient cloning

۲.۴ اجرای شبیه‌سازی

به دایرکتوری مثال سوئیچ رفتم:

در شکل ۳ میبینید که دانلود انجام شده و وارد فولدر l2-learning شدیم.

```
DHCPREQUEST for 192.168.117.128 on ens33 to 255.255.255.255 port 67 (xid=0x7a01fb97)
DHCPACK of 192.168.117.128 from 192.168.117.254 (xid=0x97fb017a)
bound to 192.168.117.128 -- renewal in 743 seconds.
p4@p4:~$ ping -c 3 google.com
PING google.com (172.217.18.14) 56(84) bytes of data.
64 bytes from fra02s19-in-f14.1e100.net (172.217.18.14): icmp_seq=1 ttl=128 time=78.4 ms
64 bytes from fra02s19-in-f14.1e100.net (172.217.18.14): icmp_seq=2 ttl=128 time=84.1 ms
64 bytes from fra02s19-in-f14.1e100.net (172.217.18.14): icmp_seq=3 ttl=128 time=77.6 ms

--- google.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 77.613/80.026/84.078/2.882 ms
p4@p4:~$ ls
mininet p4-tools
p4@p4:~$ cd p4-tools/
p4@p4:~/p4-tools$ ls
bmv2 bmv2-opt frn grpc libyang p4c p4-utils PI protobuf ptf
p4@p4:~/p4-tools$ git clone --depth 1 https://github.com/nsg-ethz/p4-learning.git
Cloning into 'p4-learning'...
remote: Enumerating objects: 698, done.
remote: Counting objects: 100% (698/698), done.
remote: Compressing objects: 100% (522/522), done.
[ 668.477881] SQUASHFS error: xz decompression failed, data probably corrupt
[ 668.478016] SQUASHFS error: squashfs_read_data failed to read block 0x28d1f58
remote: Total 698 (delta 317), reused 414 (delta 166), pack-reused 0 (from 0)
Receiving objects: 100% (698/698), 376.00 MiB | 723.00 KiB/s, done.
Resolving deltas: 100% (317/317), done.
Updating files: 100% (751/751), done.
p4@p4:~/p4-tools$ ls
bmv2 bmv2-opt frn grpc libyang p4c p4-learning p4-utils PI protobuf ptf
p4@p4:~/p4-tools$ cd p4-tools/p4-learning/examples/l2_learning
-bash: cd: p4-tools/p4-learning/examples/l2_learning: No such file or directory
p4@p4:~/p4-tools$ cd p4-learning/examples/l2_learning
p4@p4:~/p4-tools/p4-learning/examples/l2_learning$ sudo p4run-config p4app_cpu_multi.json
sudo: p4run-config: command not found
p4@p4:~/p4-tools/p4-learning/examples/l2_learning$
p4@p4:~/p4-tools/p4-learning/examples/l2_learning$
```

شکل ۳: after downloading the source code and entering the l2-learning directory

و این هم در شکل ۴ داخل فولدر را نشان میدهد

```
cd ~/p4-tools/p4-learning/examples/l2_learning
```

سپس شبیه‌سازی را با استفاده از اسکریپت پیکربندی زمان اجرای P4 راه‌اندازی کردم:

```
sudo p4run --config p4app_cpu_multi.json
```

این دستور، برنامه P4 را کامپایل کرده، توپولوژی Mininet را راه‌اندازی کرد و سوئیچ نرم‌افزاری BMv2 را شروع کرد.

```
p4@p4:~/p4-tools/p4-learning/examples/l2_learning$ ls
l2_learning_controller.py  network_digest_multi.py  p4app_cpu_multi.json  p4src
network_cpu_multi.py      network_digest.py        p4app_digest.json    README.md
network_cpu.py            p4app_cpu.json          p4app_digest_multi.json
p4@p4:~/p4-tools/p4-learning/examples/l2_learning$ _
```

شکل ۴: in the l2-learning folder

۳.۴ تأیید عملکرد

در داخل محیط خط فرمان Mininet (>mininet)، اتصال را با دستور pingall تأیید کردم. نتیجه: تمام هاست‌ها (h1, h2, ... h6) با موفقیت بسته‌ها را با صفر درصد از دست‌دادن بسته مبادله کردند. این موضوع تأیید کرد که سوئیچ به درستی آدرس‌های MAC مبدأ را یاد می‌گیرد و بسته‌ها را به پورت مقصد صحیح هدایت می‌کند.

در شکل ۵ مشاهده میکنید که اتصال به هاست‌ها به یکدیگر تأیید میشود.

```

*****
*****
Network configuration for: h6
Default interface: h6-eth0      10.0.0.6      00:00:0a:00:00:06
*****
Starting mininet CLI...

=====
Welcome to the P4 Utils Mininet CLI!
=====
Your P4 program is installed into the BMV2 software switch
and your initial configuration is loaded. You can interact
with the network using the mininet CLI below.

To inspect or change the switch configuration, connect to
its CLI from your host operating system using this command:
    simple_switch_CLI --thrift-port <switch thrift port>

To view a switch log, run this command from your host OS:
    tail -f <log_dir>/<switchname>.log
By default log directory is "./log".

To view the switch output pcap, check the pcap files in <pcap_dir>:
    for example run:  sudo tcpdump -xxx -r s1-eth1.pcap
By default pcap directory is "./pcap".

*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5 h6
h2 -> h1 h3 h4 h5 h6
h3 -> h1 h2 h4 h5 h6
h4 -> h1 h2 h3 h5 h6
h5 -> h1 h2 h3 h4 h6
h6 -> h1 h2 h3 h4 h5
*** Results: 0% dropped (30/30 received)
mininet> _

```

شکل ۵: ping all command

در شکل ۶ اتصال هاست $h1$ به $h2$ تایید شده است. با صفر درصد پکت لاس. در شکل ۷ هم برقراری اتصال به هاست $h1$ را میبینید.

```

mininet> h1 ping h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=1.16 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=4.65 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=2.36 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=4.88 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=4.22 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=5.12 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=2.87 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=3.86 ms
^C
--- 10.0.0.2 ping statistics ---
8 packets transmitted, 8 received, 0% packet loss, time 7011ms
rtt min/avg/max/mdev = 1.162/3.639/5.119/1.299 ms
mininet>

```

شکل ۶: connection of h1 to h2

```

mininet> h1 python3
Python 3.8.10 (default, May 26 2023, 14:05:08)
[GCC 9.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> h^?^?^?^?^?^?
>>> _

```

شکل ۷: h1 python

۴.۴ اتصال به سوئیچ

برای این کار، نیاز به یک ترمینال مجزا بود که در محیط متنی VM با فشردن Alt + F2 به tty2 دسترسی یافتیم. چالش اول، مسیر CLI بود؛ دستورالعمل پروژه مسیر

./tools/runtime_CLI.py

را ذکر کرده بود که در آن دایرکتوری موجود نبود. برای یافتن مسیر صحیح، از دستور find استفاده شد:

```
find /p4-tools -name runtime_CLI.py
```

خروجی نشان داد که مسیر صحیح

```
/home/p4/p4-tools/bmv2/tools/runtime_CLI.py
```

است. در نهایت، با اجرای دستور زیر، اتصال به سوئیچ برقرار شد و پرامپت RuntimeCmd نمایش داده شد:

```
sudo /home/p4/p4-tools/bmv2/tools/runtime_CLI.py --thrift-port 9090
```

۵ فاز چهارم: تحلیل سوئیچ (رابط خط فرمان زمان اجرا)

برای تأیید منطق داخلی سوئیچ، جداول ارسال آن را در زمان واقعی بازرسی کردم.

۱. دسترسی: با `Alt + F2` یک پنجره ترمینال دوم باز کردم و به صفحه کنترل سوئیچ متصل شدم:

```
sudo ./tools/runtime_CLI.py --thrift-port 9090
```

۲. بازرسی جداول:

از دستور `table_dump` برای بازرسی جداول زیر استفاده کردم:

- جدول `smac` (یادگیری آدرس MAC مبدأ): تأیید شد که سوئیچ این جدول را با آدرس‌های MAC بسته‌های ورودی پر می‌کند (یادگیری "چه کسی کجاست").
 - جدول `dmac` (ارسال مقصد): تأیید شد که سوئیچ از آدرس‌های یادگرفته‌شده برای تصمیم‌گیری ارسال استفاده می‌کند.
 - جدول `broadcast`: قوانین ارسال همگانی برای مقاصد ناشناخته تأیید شد.
- مثلاً برای `switch-info` به این شکل ۸ میتوانیم ببینیم.

```

load_new_config_file      table_dump_entry_from_key
mc_dump                   table_dump_group
mc_mgrp_create            table_dump_member
mc_mgrp_destroy           table_indirect_add
mc_node_associate         table_indirect_add_member_to_group
mc_node_create            table_indirect_add_with_group
mc_node_destroy           table_indirect_create_group
mc_node_dissociate        table_indirect_create_member
mc_node_update            table_indirect_delete
mc_set_lag_membership     table_indirect_delete_group
meter_array_set_rates     table_indirect_delete_member
meter_get_rates           table_indirect_modify_member
meter_set_rates           table_indirect_remove_member_from_group
port_add                  table_indirect_reset_default
port_remove               table_indirect_set_default
pvs_add                   table_indirect_set_default_with_group
pvs_clear                 table_info
pvs_get                   table_modify
pvs_remove                table_num_entries
register_read              table_reset_default
register_reset             table_set_default
register_write             table_set_timeout
reset_state               table_show_actions
serialize_state           write_config_to_file
set_crc16_parameters

Undocumented commands:
=====
EOF greet

RuntimeCmd: switch_info
device_id                  : 1
thrift_port                : 9090
notifications_socket       : ipc:///tmp/bmv2-1-notifications.ipc
elogsger_socket            : None
debugger_socket            : None
RuntimeCmd: _

```

شکل ۸: switch info

۱.۵ بررسی جداول P4 بخش امتیازی

در ابتدا، تلاش برای خواندن جدول با خطای TSocket read 0 bytes مواجه شد. این خطا نشان می‌داد که شبیه‌سازی Mininet (در tty1) crash کرده است. با راه‌اندازی مجدد شبیه‌سازی در tty1 و اتصال دوباره در tty2، امکان بررسی جداول فراهم شد. جداول کلیدی زیر با موفقیت بررسی شدند:

smac: این جدول برای یادگیری آدرس‌های MAC مبدأ بود و اکشن پیش‌فرض آن MyIngress.mac_learn بود.

table__dump dmac: جدول آدرس‌های MAC مقصد که در ابتدا تنها دارای اکشن پیش‌فرض NoAction بود.

table__dump broadcast: این جدول شامل ۳ رول برای مدیریت بسته‌های broadcast بود. این رول‌ها

اطمینان حاصل می‌کردند که بسته broadcast به پورتنی که از آن وارد شده، بازگردانده نشود. از طریق *runtime environment* که در شکل ۹ آمده، دستورات را وارد میکنیم.

```
act_prof_modify_member      switch_info
act_prof_remove_member_from_group table_add
counter_read                table_clear
counter_reset               table_delete
counter_write               table_dump
help                        table_dump_entry
load_new_config_file        table_dump_entry_from_key
mc_dump                     table_dump_group
mc_mgrp_create              table_dump_member
mc_mgrp_destroy             table_indirect_add
mc_node_associate           table_indirect_add_member_to_group
mc_node_create              table_indirect_add_with_group
mc_node_destroy             table_indirect_create_group
mc_node_dissociate          table_indirect_create_member
mc_node_update              table_indirect_delete
mc_set_lag_membership        table_indirect_delete_group
meter_array_set_rates        table_indirect_delete_member
meter_get_rates              table_indirect_modify_member
meter_set_rates              table_indirect_remove_member_from_group
port_add                     table_indirect_reset_default
port_remove                  table_indirect_set_default
pvs_add                      table_indirect_set_default_with_group
pvs_clear                    table_info
pvs_get                      table_modify
pvs_remove                   table_num_entries
register_read                 table_reset_default
register_reset                table_set_default
register_write                table_set_timeout
reset_state                  table_show_actions
serialize_state              write_config_to_file
set_crc16_parameters

Undocumented commands:
=====
EOF greet

RuntimeCmd:
```

شکل ۹: help command for finding proper commands

در شکل ۱۰ و ۱۱ میتوانید اطلاعات جداول مربوط به *mac* و یا *broadcast* را ببینید.

```

=====
TABLE ENTRIES
*****
Dumping entry 0x0
Match key:
* ethernet.dstAddr      : EXACT      00000a000001
Action entry: MyIngress.forward - 01
*****
Dumping entry 0x1
Match key:
* ethernet.dstAddr      : EXACT      00000a000002
Action entry: MyIngress.forward - 02
*****
Dumping entry 0x2
Match key:
* ethernet.dstAddr      : EXACT      00000a000003
Action entry: MyIngress.forward - 03
*****
Dumping entry 0x3
Match key:
* ethernet.dstAddr      : EXACT      00000a000004
Action entry: MyIngress.forward - 03
*****
Dumping entry 0x4
Match key:
* ethernet.dstAddr      : EXACT      00000a000005
Action entry: MyIngress.forward - 03
*****
Dumping entry 0x5
Match key:
* ethernet.dstAddr      : EXACT      00000a000006
Action entry: MyIngress.forward - 03
=====
Dumping default entry
Action entry: NoAction -
=====
RuntimeCmd: _

```

شکل ۱۰: for the router broadcast (forward) table(bonus)

```

=====
TABLE ENTRIES
*****
Dumping entry 0x0
Match key:
* ethernet.srcAddr      : EXACT      00000a000001
Action entry: NoAction -
*****
Dumping entry 0x1
Match key:
* ethernet.srcAddr      : EXACT      00000a000002
Action entry: NoAction -
*****
Dumping entry 0x2
Match key:
* ethernet.srcAddr      : EXACT      00000a000003
Action entry: NoAction -
*****
Dumping entry 0x3
Match key:
* ethernet.srcAddr      : EXACT      00000a000004
Action entry: NoAction -
*****
Dumping entry 0x4
Match key:
* ethernet.srcAddr      : EXACT      00000a000005
Action entry: NoAction -
*****
Dumping entry 0x5
Match key:
* ethernet.srcAddr      : EXACT      00000a000006
Action entry: NoAction -
=====
Dumping default entry
Action entry: MyIngress.mac_learn -
=====
RuntimeCmd:

```

شکل ۱۱: for the router mac table(bonus)

۶ نتیجه‌گیری

این پروژه با موفقیت پیاده‌سازی و استقرار یک سوئیچ لایه ۲ مبتنی بر P4 را نشان داد. دستاوردهای یادگیری اصلی تنها به نحو P4 محدود نمی‌شد، بلکه شامل مهارت‌های گسترده‌ای در مدیریت سیستم نیز بود: مدیریت دیسک‌های مجازی، بازیابی از خرابی اسنپ‌شات‌ها و پیکربندی دستی زیرسیستم‌های شبکه لینوکس. شبیه‌سازی نهایی تأیید کرد که منطق سوئیچ یعنی یادگیری آدرس‌های MAC مبدأ و ارسال همگانی بسته‌های ناشناخته به درستی عمل می‌کند. [۱] و [۲]

مراجع

- [1] “Computer networks,” https://gaia.cs.umass.edu/kurose_oss/ninth.php.
- [2] “Gemini ai interactive assistance,” <https://gemini.google.com/app/c27f7d3d82f1862e>.